

1. Identities equivalent to $\text{atan}(1)$

A demonstration that

$$8 \text{atan} \frac{1}{10} - \text{atan} \frac{1}{239} - 4 \text{atan} \frac{1}{515} = \text{atan} 1 \quad (1.1)$$

First, recall the sum-of-angles identity for the tangent function:

$$\tan(x + y) = \frac{\tan x + \tan y}{1 - (\tan x)(\tan y)} \quad (1.2)$$

Furthermore, by the nature of inverse functions,

$$x = \text{atan}(\tan x) = \tan(\text{atan} x) \quad (1.3)$$

Therefore:

$$\begin{aligned} \text{atan} x + \text{atan} y &= \text{atan}(\tan(\text{atan} x + \text{atan} y)) \\ &= \text{atan} \frac{\tan(\text{atan} x) + \tan(\text{atan} y)}{1 - \tan(\text{atan} x) \tan(\text{atan} y)} \\ &= \text{atan} \frac{x + y}{1 - xy} \end{aligned} \quad (1.4)$$

And, as a special case,

$$\begin{aligned} 2 \text{atan} x &= \text{atan} x + \text{atan} x \\ &= \text{atan} \frac{2x}{1 - x^2} \end{aligned} \quad (1.5)$$

Thus:

$$\begin{aligned} 8 \text{atan} \frac{1}{10} &= 4 \text{atan} \frac{2/10}{1 - 1/100} = 4 \text{atan} \frac{20}{99} \\ &= 2 \text{atan} \frac{40/99}{1 - 400/9801} = 2 \text{atan} \frac{3960}{9401} \\ &= \text{atan} \frac{7920/9401}{1 - 15681600/88378801} = \text{atan} \frac{74455920}{72697201} \end{aligned} \quad (1.6)$$

And

$$\begin{aligned} 4 \text{atan} \frac{1}{515} &= 2 \text{atan} \frac{515}{132612} \\ &= \text{atan} \frac{136590360}{17585677319} \end{aligned} \quad (1.7)$$

So

$$\begin{aligned} 8 \text{atan} \frac{1}{10} - \text{atan} \frac{1}{239} - 4 \text{atan} \frac{1}{515} &= \text{atan} \frac{74455920}{72697201} - (\text{atan} \frac{1}{239} + \text{atan} \frac{136590360}{17585677319}) \\ &= \text{atan} \frac{74455920}{72697201} + \text{atan} \frac{-1758719}{147153121} \\ &= \text{atan} 1 \quad \blacksquare \end{aligned}$$

Another identity that could be used is:

$$\operatorname{atan} 1 = 4 \operatorname{atan} \frac{1}{5} - \operatorname{atan} \frac{1}{239} \quad (1.8)$$

This has the advantage of only requiring two passes, but has the disadvantage that the series for $1/5$ converges more slowly than those for $1/10$ and $1/515$ combined. The proof of correctness may be obtained in a manner very similar to the one shown above, and will not be spelled out here.

[Note that these Taylor-series based computations are not state-of-the-art for computing absurd quantities of digits of tau or pi; the trillion-plus digit record holders use a hypergeometric series developed by the brothers David and Gregory Chudnovsky, which cranks out about 15 digits per term computed for the series. Another approach involves refinements of the Gauss-Legendre algorithm, such as one by Richard Brent and Eugene Salamin (1976), which *doubles* the number of accurate digits with each iteration. But these approaches are not as easy to understand or implement as Taylor-series based approaches like the one used in this program.]

Putting this all together and running calculations using multi-precision arithmetic with “digit”s of base **BASE** is how this code accomplishes its task.

2. Error Analysis

Let

$$f(x, n) = \frac{x^{2n+1}}{2n+1} \quad [|x| \leq 1] \quad (2.1)$$

Then the Taylor-Maclaurin series for $\text{atan } x$ (over $|x| \leq 1$) can be written as:

$$\text{atan } x = \sum_{i=0}^{\infty} (-1)^i f(x, i) \quad (2.2)$$

Recall that if a_i is a monotonically non-increasing sequence with $\lim_{i \rightarrow \infty} a_i = 0$, then the sum $\sum_{i=0}^{\infty} (-1)^i a_i$ converges; truncating this sum after term n will have an error whose magnitude is at most a_{n+1} . Thus for $\text{atan } x$, provided that $0 \leq x \leq 1$, the error incurred by truncating the series after the n -th term is:

$$\text{err}(n) \leq f(x, n+1) = \frac{x^{2n+3}}{2n+3} < \frac{x^{2n+1}}{2n+1} = f(x, n) \quad (2.3)$$

We will restrict ourselves to consideration of positive x (i.e., $0 < x \leq 1$, or $1 \leq 1/x$) and assume that $a = 1/x$ is the worst-case choice for the relations being considered. Substituting into (2.3) gives

$$\text{err}(n) < f(1/a, n) \quad (2.4)$$

We want our error term $\text{err}(n)$ to be less than B^{-D} , for some specified base B and to-be-determined base- B -digit-count D :

$$\text{err}(n) < B^{-D} \quad (2.5)$$

Furthermore, we want to fit intermediate results into an integer which cannot exceed some maximum $I_{\max}$. Each word of our multi-precision vector is modulo-reduced by $2n+1$ on outer-loop iteration n . Each step computes

$$r_i B + t_i \leq I_{\max}$$

at word-index i , for carry-down residue r_i and previous-term t_i , each of which is in the range $0 \leq \star \leq 2n+1-1 = 2n$, thus the worst-case intermediate result is $2nB + 2n \leq I_{\max}$. Solving for n yields:

$$n \leq \frac{I_{\max}}{2(B+1)} \quad (2.6)$$

Our goal is to find the largest D that simultaneously satisfies relations (2.4), (2.5), and (2.6). It is acceptable to find a value D' that is less than the actual maximum, if an approximation makes life easier, but we must never allow the resultant D' to be bigger than D and we prefer not to underestimate D by *too* much.

Now equations (2.4) and (2.5) do not have a natural ordering, but given our goal of wanting to derive a “safe” value of D , we choose to combine them with this ordering:

$$\text{err}(n) < f(1/a, n) < B^{-D} \quad (2.7)$$

For brevity, let

$$R = \frac{I_{\max}}{B+1} \quad (2.8)$$

Combining substitutions of (2.6) and (2.8) into equation (2.7) gives:

$$f(1/a, R/2) < B^{-D}$$

Solving for D yields:

$$D < \frac{-\log f(1/a, R/2)}{\log B} \quad (2.9)$$

Using $(1/a)^x = a^{-x}$, $\log x / \log B = \log_B x$, and the definition of f given in equation (2.1):

$$D < \frac{-\log \frac{a^{-(2(R/2)+1)}}{2(R/2)+1}}{\log B} = (R+1) \log_B a + \log_B(R+1) \quad (2.10)$$

Converting the digit count to base-10 digits involves multiplying by $\log_{10} B$. Because

$$(\log_B x)(\log_{10} B) = \frac{\log_{10} x}{\log_{10} B} \log_{10} B = \log_{10} x$$

we can determine that the constraint on the number of decimal digits d as

$$d = D \log_{10} B < (R+1) \log_{10} a + \log_{10}(R+1) \quad (2.11)$$

To take a concrete example, let us assume that our worst-case is $a = 5$ (as when evaluating the relation in equation (1.8)), that $B = 10^{12}$, and that $I_{\max} = 2^{63} - 1$; then $R = \frac{2^{63}-1}{10^{12}+1}$ and we get

$$d < \left(\frac{2^{63}-1}{10^{12}+1} + 1 \right) \log_{10} 5 + \log_{10} \left(\frac{2^{63}-1}{10^{12}+1} + 1 \right) \approx 6\,446\,868$$

So for these parameters we want to ensure that no more than 6 446 868 decimal digits are allowed to be computed and printed, in order to avoid the risk of errors from integer overflow (whether they are silent as in C or panicking as in Rust).

Some further adjustments:

- To re-iterate, we don’t need to be precise in our upper bound; lowering by a few digits is acceptable.
- Hence it is okay to drop the $\log_{10}(R+1)$ term in equation (2.11); even in the situation with $I_{\max} = 2^{64}$ and $B = 10$, which will give a fairly large $R = \frac{I_{\max}}{B+1}$, we would lose fewer than 20 digits in our estimation of d .
- We are assuming that our worst case happens with $a = 5$; $\log_{10}(5) > 339/485$ with an error in the ballpark of one part per million.

- To minimize user confusion, we want to round the “maximum” down to the nearest multiple of $\log_{10} B$, since we will always be outputting an integer number of base- B “digits”.

All of this can be accomplished with integer-only calculations by using the following Rust function. Here `Xword` is the signed-integer type used for calculations, `Xword::MAX` represents $I_{\max}$, `WORDDIGITS` = $\log_{10} B$; we require that `WORDDIGITS` be an integer, hence that we end up with `BASE` = B below:

```
const BASE: Xword = (10 as Xword).pow(WORDDIGITS as u32);
fn max_digits() -> Xword {
    let d = (Xword::MAX / (BASE+1) + 1) * 339 / 485;
    d - d % (WORDDIGITS as Xword)
}
```

Note that if we use $a = 10$ (such as when using equation (1.1)) instead of $a = 5$ (as with equation (1.8)), then instead of using $\log_{10} 5 \approx 339/485$ we would use $\log_{10} 10 = 1$, which both simplifies the equation and allows for the safe computation of more digits. On the other hand, an attempt to compute $\text{atan}(1)$ directly using the Taylor series would involve multiplying by $\log_{10} 1 = 0$ — thus we would be limited to the fairly small quantity of $\log_{10}(R+1)$ digits accurately computed before encountering integer overflow problems in some intermediate calculation; this is one of the reasons we seek and use a mathematically equivalent calculation which is more numerically stable (the other reason is that even if we had infinite-precision variables, the rate of convergence for a raw $\text{atan}(1)$ is much too slow).